Model Checking Concurrent Programs with SPIN

RW796 Software Verification and Analysis

Gabriella Scott
25935453

September 1, 2026

Introduction

This report presents Promela models of two concurrent programs and the verification of their correctness properties using Spin. The first program is a first-come first-served (FCFS) process scheduler, one of three policies supported by a shared-memory process manager. A team of OpenMP threads dispatches processes from a common ready queue, moves them to a waiting queue when a resource is unavailable, and updates a shared resource table, with nested locks protecting these structures and new processes arriving while the threads are already running. The second program is a distributed Othello engine running a parallel alpha-beta search. An MPI master hands candidate moves out to worker processes as they report back, propagates a shared alpha bound between them, and reduces the search depth partway through a round when the turn clock runs down. It communicates over three channels: MPI collectives to start a round, point-to-point messages to distribute and collect work, and a socket to the referee.

The models are intentionally small. Parts of each program were omitted, some behaviour was replaced with nondeterministic choices, and the remaining parameters were fixed to small values. For these abstractions, one must assume that omitted behaviour does not affect the properties being checked. In contrast, the smaller parameter values limit the scope of a passing result but do not weaken a counterexample, since a counterexample still represents a valid execution. Each abstraction is justified when introduced, while Section **??** and Section **??** discuss the resulting limitations.

The analysis found defects in both programs. For the scheduler, 6 of the 7 checks failed, with the counterexamples tracing back to two faults in `manager.c`: a resource request that does not check whether the requester already owns the resource, and a dispatch loop that accesses shared process state without holding a lock. For the Othello engine, 4 of the 5 checks failed, with 3 faults identified in `my_player.c`, namely an initialiser escaping from the minimax minimising branch as if it were a score, comparisons between scores calculated at different search depths, and two referee message types for which the master loop has no corresponding branch. The last fault did not occur in any recorded match, illustrating the value of model checking in finding behaviours that testing may not expose.

Implementation under analysis (Program 1): **[TODO: repo URL]**
Implementation under analysis (Program 2): **[TODO: repo URL]**
Promela models, properties and verification records: `https://github.com/Gabriella-Scott/PA_Assignments`

1 Program 1: FCFS Process Scheduler

1.1 The Promela Model

This model is an abstraction of the FCFS scheduler. The original code exists in `manager.c` together with the functions it reaches:

- `schedule_fcfs`
- `execute_instr`
- `request_resource`
- `release_resource`
- `enqueue_pcb`
- `dequeue_pcb`
- `terminate`

It consists of an `init` process that seeds the ready queue, three inlines that stand for `terminate`, `request_resource` and `release_resource`, and a single `Worker` proctype instantiated once per scheduling thread. The other two policies (`schedule_rr` and `schedule_priority`) are not modelled. All three sit on the same queues, the same locks and the same resource table, so including them would have multiplied the state space without reaching a different class of defect.

The size of the model is fixed by 4 constants; each of them is the smallest value that shows the interesting behaviour. `NUM_WORKERS` is 2, because a single thread cannot interleave with anything and every race the scheduler can have needs two threads at minimum. `NUM_PROCS` is 3, because 2 processes are not enough to keep the ready queue occupied; with one running and one blocked the queue is empty, and a third is needed before a process can be dispatched while another is already blocked. `NUM_RESOURCES` is 2; this is the smallest number that presents a circular wait, since 1 process holding a resource and requesting the same one is a different fault from 2 processes each holding what the other wants [2]. `MAX_INSTR` is 3, which allows a process to request a resource, release it, and request again, so that a release can wake a blocked process and that process can then make progress.

Process and resource identifiers start at 1 and not 0. The implementation uses `NULL` to mean both "no process holds this resource" and "this process is not waiting", and reserving 0 for that purpose keeps the same convention in the model, so that `resource_owner[r] == 0` reads exactly as `resource->allocated == NULL` does.

Only `MAX_INSTR` is wrapped in an `#ifndef` guard. It is the one parameter that is varied between runs. Guarding it allows a different configuration to be supplied with `spin -D` rather than by editing the model. This ensures that every result in Section ?? comes from the same committed file.

Listing 1: Dequeue in `schedule_fcfs`

```
#pragma omp critical
{
  omp_set_nest_lock(&q_locks[0]);
  running_p = dequeue_pcb(&readyq);
  omp_unset_nest_lock(&q_locks[0]);
}
```

Listing 2: The same step in the model

```
\todo{paste the atomic dequeue block from model.pml}
```

Table 1: Mapping between the implementation and the model.

manager.c	Model	Reason
Thread in schedule_fcfs	active [NUM_WORKERS] proctype Worker	Parallel region runs the same threads over the same loop, so one proctype replicated twice captures every thread the scheduler creates.
readyq, waitingq	Buffered channels ready_q, waiting_q	FIFO order is the only behaviour of the linked lists that FCFS depends on, so buffered channels provide a direct abstraction. waiting_q also carries the resource id, which the implementation normally obtains from the instruction record.
terminatedq	pcb_state[] == TERMINATED	Nothing gets dequeued from terminatedq, but it is appended to and counted; this means membership carries no information the state field does not already hold.
pcb_t.state	pcb_state[]	Direct, indexed by process id instead of reached through a pointer.
next_instruction	instr_count[]	The instruction list becomes a bounded counter, with next_instruction == NULL represented by instr_count == MAX_INSTR. The type and target of each instruction are chosen nondeterministically rather than read from the list.
resource_t.allocated	resource_owner[]	The resource list searched by strcmp becomes an array indexed by id, and allocated == NULL becomes owner 0.
q_locks[0], omp critical, i_lock, r_lock	atomic blocks	Mutual exclusion (Mutex) is the only thing the locks provide here, and atomic supplies it without modelling lock acquisition itself [1]. Only the regions the implementation actually protects are made atomic.
terminate()	check_terminate inline	The same test, counting waiting and terminated processes and requiring an empty ready queue, evaluated over the state array rather than by walking the queues.

1.2 Correctness Properties

`pcb_mutex`: mutual exclusion over process control blocks

$$\Box \bigwedge_{p=1}^{3} executing[p] \leq 1$$

```
#define one_worker_per_pcb (executing[1] <= 1 && executing[2] <= 1 &&
    executing[3] <= 1)
ltl pcb_mutex { [] one_worker_per_pcb }
```

[TODO: Justification.]

`no_self_wait`: no process waits for a resource it owns

$$\Box \bigwedge_{p=1}^{3} (waiting\_for[p] = 0 \vee resource\_owner[waiting\_for[p]] \neq p)$$

[TODO: Promela form and justification.]

`waiting_consistent`: the waiting state is consistent

[TODO: Formula, Promela form and justification.]

`no_dup_ready`: ready queue integrity

[TODO: Formula, Promela form and justification.]

`all_terminate`: every process eventually terminates

$$\Diamond \bigwedge_{p=1}^{3} pcb\_state[p] = TERMINATED$$

[TODO: Justification.]

`ready_dispatched`: a ready process is eventually dispatched

$$\Box(pcb\_state[1] = READY \rightarrow \Diamond\, pcb\_state[1] = RUNNING)$$

[TODO: Justification.]

Deadlock

[TODO: Discussion.]

1.3 Verification Results and Analysis

[TODO: Setup paragraph.]

The double dispatch race

[TODO: Analysis and fix.]

Self deadlock in `request_resource`

[TODO: Analysis and fix.]

Table 2: Verification results, Spin 6.5.2. Depths and state counts as reported by `pan`.

Property	Config	Result	Depth	States
`pcb_mutex`	`MAX_INSTR 3`	violated	901	65 957
`no_self_wait`	`MAX_INSTR 3`	violated	379	188
`waiting_consistent`	`MAX_INSTR 3`	violated	962	69 699
`no_dup_ready`	`MAX_INSTR 2`	holds	n/a	20 861 451
`all_terminate`	`MAX_INSTR 3, -f`	violated	1069	493
`ready_dispatched`	`MAX_INSTR 3, -f`	violated	1088	11 372 991
deadlock	`MAX_INSTR 3, -DNOCLAIM`	violated	504	67 872

Liveness and fairness

[TODO: Analysis.]

The invalid end state

[TODO: Analysis.]

The property that holds

[TODO: Analysis.]

2 Program 2: [TODO: name]

2.1 The Promela Model

[TODO:]

2.2 Correctness Properties

[TODO:]

2.3 Verification Results and Analysis

[TODO:]

A Reproducing the verification results

Listing 3: Build

```
spin -a model.pml
gcc -O2 -o pan pan.c
gcc -O2 -DNOCLAIM -o pan_noclaim pan.c
```

Listing 4: Verification runs

```
./pan -a -N pcb_mutex
./pan -a -N no_self_wait
./pan -a -N waiting_consistent
./pan -a -N all_terminate    -f -m50000
./pan -a -N ready_dispatched -f -m50000 -w26
./pan_noclaim
```

```
spin -a -DMAX_INSTR=2 model.pml
gcc -O2 -o pan pan.c
./pan -a -N no_dup_ready -m20000 -w26
```

[TODO: State the commit hash the results were produced from.]

References

[1] Gerard J. Holzmann. Promela reference: `atomic`(3). `https://spinroot.com/spin/Man/atomic.html`. Accessed 2025.

[2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. Wiley, 10th edition, 2018.